

Technical Paper

Mathematical model and adaptive simulated annealing algorithm for mixed-model assembly job-shop scheduling with lot streaming

Lixin Cheng^{a,b}, Qiuhua Tang^{a,b,*}, Liping Zhang^{a,b}

^a Key Laboratory of Metallurgical Equipment and Control Technology of Ministry of Education, Wuhan University of Science and Technology, China

^b Hubei Key Laboratory of Mechanical Transmission and Manufacturing Engineering, Wuhan University of Science and Technology, China

ARTICLE INFO

Keywords:

Mixed-model assembly job-shop scheduling
Lot streaming
Adaptive simulated annealing algorithm
Rule
Q-learning

ABSTRACT

Mixed-model assembly and lot streaming are two key technologies to improve efficiency and reduce setups in a multi-variety and small-batch discrete manufacturing environment. Yet few studies consider the two technologies in assembly job-shop scheduling problems. Thus, this paper investigates the mixed-model assembly job-shop scheduling problem with lot streaming, which contains two closely coordinated stages with differentiated production lot size: the processing stage with lot streaming and the mixed-model assembly stage with single-piece flow. To tackle this problem, a mathematical model including new constraints i.e. production sequence constraints within/among sublots, sequence-dependent setup constraints between sublots and mixed-model assembly constraints, is developed to establish the temporal and spatial links between the processing and assembly stages. Besides, an adaptive simulated annealing algorithm is proposed. In this algorithm, a sublot sequence rule considering sublot and assembly features is discovered via gene expression programming to promote the performance of initial solutions; a Q-learning-based temperature control mechanism is designed to adaptively adjust the acceptance probability of inferior solutions and accordingly avoid local optima; an adaptive selection mechanism of neighborhood operators is designed to increase the chance of finding potential solutions; an adaptive adjustment mechanism of neighborhood size is put forward to balance computing resources and the coverage of neighborhood space. Experimental results show that both the discovered rule and the adaptive mechanisms are effective, and the developed algorithm is more adapt to fix the studied problem than other state-of-the-art algorithms.

1. Introduction

With the growing demand for personalized products, the manufacturing pattern of structural parts such as bogies and moulds is changing from mass production to multi-variety and small batch production. These structural parts adopt discrete manufacturing which contains two stages: processing and assembly. The processing stage focus on manufacturing a wide variety of components with specific processing routes in the form of job-shop production. The assembly stage forms the finished products by joining the components of each product together. At present, most studies assumed that the assembly stage is implemented via a single machine or parallel machines [1], and ignored the fact that mixed-model assembly lines are extensively adopted on the site to achieve the collinear assembly of diverse products [2]. To meet the practical needs, it is urgent to carry out in-depth research on the

mixed-model assembly job-shop scheduling problems.

To the best of our knowledge, only one research has addressed the mixed-model assembly job-shop scheduling problem [3]. Moreover, some practical factors such as setups and lot streaming are not considered in this research. In fact, setup times exist as long as two successive operations from different types of components are performed on a machine. Since a wide variety of components are performed in the processing stage, numerous setup times are required. An efficient method to reduce setup times is to treat all components belonging to the same type as a lot and process them continuously. However, when the lot size is large, this method not only greatly limits the production flexibility, but also seriously slows down the flow of production [4]. As a result, some defects are incurred such as lower production efficiency and higher work-in-process inventory. In this context, lot streaming, a method to split a production lot into smaller sublots, needs to be introduced to fix

* Corresponding author at: Key Laboratory of Metallurgical Equipment and Control Technology of Ministry of Education, Wuhan University of Science and Technology, China.

E-mail address: tangqiuhua@wust.edu.cn (Q. Tang).

<https://doi.org/10.1016/j.jmansys.2023.08.008>

Received 11 April 2023; Received in revised form 11 July 2023; Accepted 15 August 2023

Available online 29 August 2023

0278-6125/© 2023 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

the above defects. Thus, the mixed-model assembly job-shop scheduling problem with lot streaming (MAJSP-LS) is investigated in this work.

For the studied MAJSP-LS, the processing stage applies lot streaming, while the assembly stage adopts single-piece flow to move products one by one from the start to the end of the assembly line. The differentiated production lot size leads to a challenge in establishing the temporal and spatial links between the two stages. In addition, the studied problem contains three coupled decisions: subplot split, subplot sequence and assembly sequence. As the subplot split changes, the subplot sequence must change accordingly. And, due to the assembly relationship, the assembly sequence is seriously affected by the subplot sequence. To tackle this novel problem, a new mixed-integer linear programming model (MILP) and an adaptive simulated annealing algorithm are developed. The main contributions of this work are listed below.

- (1) The proposed model not only considers the subplot split, subplot sequence and sequence-dependent setup constraints but also establishes the sequence constraints of components within a subplot and mixed-model assembly constraints.
- (2) Problem-specific features i.e. subplot and assembly features, are extracted and fed into the gene expression programming (GEP) to construct an efficient subplot sequence rule.
- (3) Three self-adaptive mechanisms are designed. The temperature is adaptively controlled by Q-learning according to the evolution state to avoid local optima. Neighborhood operators are adaptively selected according to their historical performances to increase the chance of finding potential solutions. Neighborhood size changes adaptively with the running state to balance computing resources and the coverage of neighborhood space.

The remainder of this work is organized below. Section 2 is the literature review. Section 3 shows the motivation example. Section 4 formulates the MILP model. Section 5 describes the proposed ASA. Section 6 is the experimental study. Section 7 draws a conclusion.

2. Literature review

This section first presents a review of the assembly job-shop scheduling problems and later reports the integration of lot streaming and assembly job-shop scheduling.

2.1. Assembly job-shop scheduling

Structural parts are generally assembled from a set of components, and each one of these components needs to be processed before assembly [5]. The scheduling problems considering both processing and assembly stages are called assembly scheduling problems. As stated in [1], assembly scheduling problems can be represented by $\alpha_1 \rightarrow \alpha_2 | \beta | \gamma$. Here, β represents attributes, such as setup. γ is optimization objective. α_1, α_2 respectively denotes the machine layout in the processing and assembly stages. α_1 can be a single machine [6], parallel machines [7], flow shop [8], job-shop (Jm) [9] and so on. In general, if each component has a specific processing route, Jm layout is adopted and the resulting assembly scheduling problems $Jm \rightarrow \alpha_2$ are classified as assembly job-shop scheduling problems (AJSP).

In the assembly stage of AJSP, α_2 can also be a single machine (1), parallel machines (Pm) and flow shop (Fm). Among them, the single-machine layout is the simplest case, in which all assembly operations are performed on one machine. In this regard, Wong et al. [10] studied the AJSP consisting of one inventory station and one assembly station. All processed lots were stored in the inventory station, and the assembly of these lots started if at least one unit of the final product can be produced. Fattahi et al. [11] addressed the flexible AJSP and a hybrid algorithm was proposed to optimize the processing machine selection and production sequence of processing and assembly. Wu et al. [12] addressed the distributed AJSP, in which each assembly factory

contained one machine for assembly. Compared with the single machine layout, the Pm layout is more flexible but more complex due to the involvement of assembly machine allocation. For the assembly stage with Pm layout, Wang et al. [13] studied the AJSP with uncertain assembly time via a dual Q-learning, in which the top-level Q-learning determined the machine allocation and the bottom-level optimized the sequence schedule. Zhang et al. [9] addressed the flexible AJSP considering sequence-dependent setups in the processing stage. Ren et al. [14] investigated the energy-aware flexible AJSP. Fatemeh et al. [15] and Zhang et al. [16] extend the flexible AJSP by introducing lot streaming. Different from the $1/Pm$ layout which only considers the assembly operation of components, the existing research of assembly scheduling with Pm layout may further consider the collection, transport, painting, inspection, etc [17]. Among them, the most studied case is the $F2$ layout which consists of transport and assembly phases [18].

In the above studies, assembly operations are set to be performed on a single machine or one of the parallel machines, which is unrealistic. In fact, it is common to witness that mixed-model assembly is adopted in the assembly workshop [2]. In this scenario, components of each product are assigned to multiple stations and assembled on these stations according to flow production. The processing process of corresponding components can be controlled and pulled according to the demands of each assembly station. As a result, high production efficiency and low work-in-process inventory are achieved. To date, only two studies had comprehensively investigated the assembly scheduling with mixed-model assembly. The first one is by [19], where components were first processed on dedicated parallel machines, and then assembled on a set of assembly stations in a flow shop. Different from the first one, Cheng et al. [20] investigated the mixed-model assembly job-shop scheduling problem, in which the mixed-model assembly was applied for forming products having a single-level assembly structure and the job-shop production was adopted for processing the required components having specific processing routes. However, in the above two research, none of them considered the lot streaming scheduling of components.

2.2. Integration of lot streaming and assembly job-shop scheduling

Lot streaming (LS) aims to split production lots into smaller sublots so that operations of sublots from a lot can be overlapped to improve production efficiency [21,22]. If the subplot size and subplot quantity change throughout the process, the sublots is called variable sublots; otherwise, consistent sublots. Particularly, if all consistent sublots have identical subplot size, we call them equal and consistent sublots. Note that, variable sublots was too flexible to be applied to real-world production [23], while consistent sublots especially the equal and consistent sublots, had a wide field of applications and hence received much attention. In addition, sublots from different lots can be processed with intermingling or with non-intermingling. For the sublots with non-intermingling, it demands that all sublots of a lot are processed continuously, which may miss some better global scheduling schemes [24]. Based on the above analysis, the equal and consistent sublots with intermingling is considered in this work.

By now, LS has been successfully applied to various scheduling problems. Here, the applications of LS to job-shop scheduling problems (JSP) and AJSP are discussed. The JSP-LS includes two subproblems: subplot split and sequence. In most studies, the two subproblems are solved separately. For example, Chan et al. [25] investigated the JSP considering equal sublots and variables sublots. In this work, subplot split and sequence were encoded separately and sequentially solved. Buscher et al. [26] solved the JSP with consistent sublots via a three-phase method. The first phase determined initial subplot sizes, then tabu search was designed to determine a sequence schedule. The last phase further adjusted subplot size. Lei et al. [27] addressed the JSP with consistent sublots and transportation, and solved it under the given subplot sizes. Božek et al. [28] addressed the flexible JSP with variable

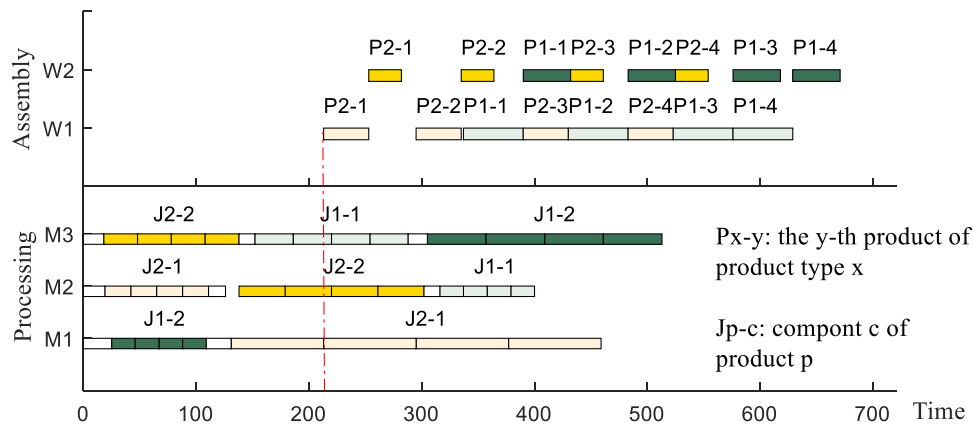


Fig. 1. The scenario of MAJSP without LS.

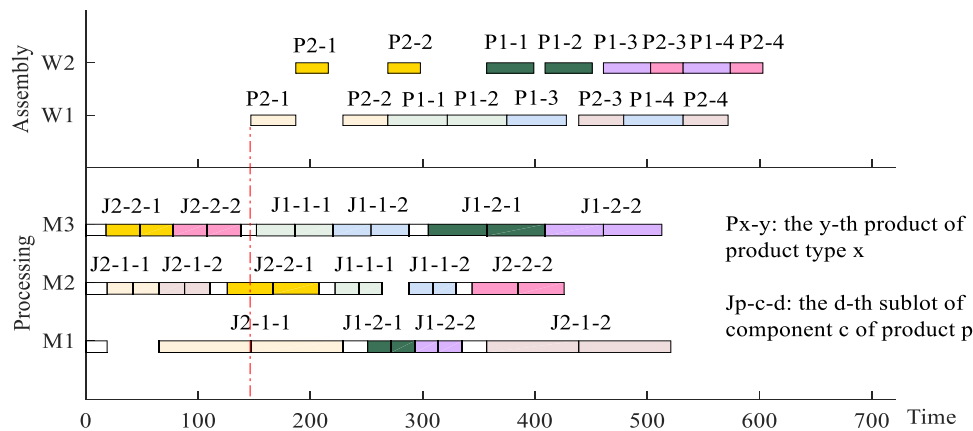


Fig. 2. The scenario of MAJSP with LS.

sublots and non-intermingling. In this work, the sequence and machine allocation were first optimized under the given smallest sublots, and then subplot sizes were maximized without extending the makespan. Although optimizing subplot split and sequence separately can simplify the studied problem, the global optimum may not be guaranteed. Given this, Xu et al. [29] addressed the flexible JSP with variable sublots, in which the subplot split and sequence were simultaneously encoded and optimized. Defersha et al. [30] and Vuong et al. [31] developed a novel encoding in which the subplot split segment considered the maximum and the actual number of sublots. In this way, the created sequence segment could remain feasible for any subplot split scheme.

As an extension of JSP-LS, the AJSP-LS arises when the processed sublots need to be assembled to form final products. Chan et al. [32] appeared to be the first to apply LS to AJSP. The authors solved it by a genetic algorithm-dispatching rule method, in which genetic algorithm was applied to determine the LS condition, and simple dispatching rules were applied to create a processing sequence. Later, Chan et al. [33] and Wong et al. [10] extended the above study by allowing part sharing in the assembly stage, and LS condition and processing sequence were simultaneously optimized. Li et al. [4] studied the flexible AJSP with LS, and three decisions: LS condition, subplot sequence and machine assignment were considered. Zhang et al. [16] investigated the flexible AJSP with LS, part sharing and setup. In this work, each subplot denoted a batch of identical parts, and a distributed ant colony system was designed to determine machine allocation and subplot sequence. In the above works, assembly was treated as a special machining operation, or performed by checking the processing state of lots. If at least one unit of the final product can be produced, these lots were sent for assembly. In this case, the AJSP-LS works very similarly to the JSP-LS. Different from the above research, Fatemeh et al. [15] applied LS to both the machining

and assembly stages of the flexible AJSP. The authors considered not only the subplot split of components, processing sequence, but also the subplot split of product, and assembly sequence. But, due to the complexity, the LS and sequence scheduling problems were optimized hierarchically.

To sum up, the existing studies ignore the impact of LS on the mixed-model assembly of products. The integrated optimization of component processing with lot streaming and mixed-model assembly has not been comprehensively investigated.

2.3. Optimization approach

With the integration of mixed-model assembly and LS, the studied MAJSP-LS is far more complex than the classical JSP, a well-known NP-hard problem. To solve these complex scheduling problems, numerous heuristic rules and meta-heuristic algorithms are developed. Regarding the heuristic rules, Panwalker and Iskander [34] summarized over one hundred scheduling rules including simple priority rules, the combination of simple priority rules, weighted priority indexes, and so on. Although heuristic rules can quickly create a feasible solution and are convenient implementation in real practice, their performance for a new scheduling problem is hard to be guaranteed since they are extremely sensitive to the problem attributes and optimization objectives [35]. In this situation, rule mining technologies including GEP and genetic programming were widely applied to design problem-specific rules. Typically, Zhang et al. [35] applied multi-gene GEP to construct rules for energy-efficient JSP. Zhang et al. [36] provided a systematic review of genetic programming for production scheduling. In this book, the basics of genetic programming, as well as advanced topics including the handling of multiple objectives and multitask genetic programming

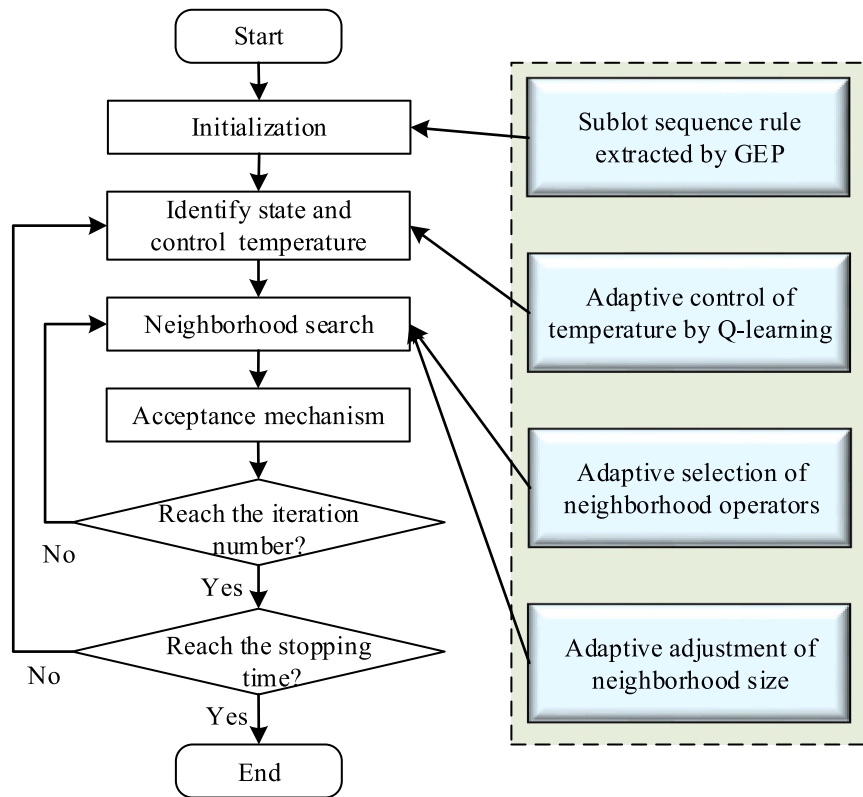


Fig. 3. Framework of the ASA.

were demonstrated. In fact, designing rules via GEP/genetic programming is time-consuming and the performance of the designed rule is generally affected by many factors such as the features selected for constructing rules and the quality of the training dataset. To solve this, ensemble methods were used to combine a set of rules generated by gene programming to improve the performance [37]. In addition, machine learning methods such as feature selection had been introduced into GEP/genetic programming to improve the effectiveness of rule mining. A review of heuristic design in JSP via genetic programming and machine learning can be found in the work by Zhang et al. [38].

As for the meta-heuristic algorithms, they can be classified as swarm-based algorithms such as genetic algorithm, and single solution-based algorithms such as simulated annealing algorithm (SA). Regarding the assembly job-shop scheduling problems, Guo et al. [39] designed a learning interactive genetic algorithm for the assembly job-shop scheduling problem. Sukkerd and Wuttipornpun [40] provided a hybrid genetic algorithm and tabu search for the flexible assembly job-shop scheduling problem. Cheng et al. [3] proposed an enhanced SA for the MAJSP. Regarding the LS scheduling problems, Güçdemir and Selim [41] designed a SA-based simulation optimization method for job-shop scheduling considering lot splitting and customer relationship management. Bozek et al. [28] applied tabu search to solve the flexible JSP with LS. Li et al. [42] solved the energy-efficient flexible JSP with LS via a hybrid algorithm combining the imperialist competitive algorithm and SA. For the integration of assembly job-shop scheduling with LS, Chan et al. [32] designed a genetic algorithm-dispatching rule method for the assembly job-shop scheduling with LS. Fatemeh et al. [15] developed a variable neighborhood search algorithm for the flexible assembly job-shop scheduling with LS. A similar problem was solved by Li et al. [4] via an improved artificial bee colony algorithm. Sun et al. [43] developed a teaching-learning-based optimization for the fuzzy flexible assembly job-shop scheduling with batch splitting. Although numerous meta-heuristic algorithms have been developed, their outstanding performance in solving a specific problem cannot be

separated from the in-depth mining of problem features, the design of problem-specific operators, and the control and optimization of key parameters.

In sum, both the heuristic rules and meta-heuristic algorithms have some inherent strengths and weaknesses. Considering the lack of prospective research on the MAJSP-LS and the lack of insufficient exploration of the correlation features between lot streaming scheduling and mixed-model assembly, it's hard to construct superior rules that fully adapt to the MAJSP-LS yet. In this case, we try to use GEP to mine a preliminary combination rule considering sublot and assembly features and use the designed rule to provide a high-quality initial solution. Further, an enhanced SA is adopted to guide the evolution of the obtained initial solution, since SA has few parameters, simple structure, and strong local search ability, and it has been successfully applied to solve both the MAJSP and LS scheduling problem in the latest research.

3. Motivation example

The original motivation for the studied problem comes from the assembly scheduling of bogies in a railway vehicle manufacturing industry in China. In this industry, multiple types of railway vehicles such as urban rail vehicles and high-speed EMU trains are manufactured. Each railway vehicle contains twelve/sixteen bogies, and each bogie consists of a group of components including the frame, wheelset, axle box and so on. Each bogie is manufactured through two main stages: the processing stage and the assembly stage. In the processing stage, components of different types of bogies are performed on several machines. To reduce setups, multiple components from the same type of bogie are grouped into a batch and performed continuously on the site. In the assembly stage, different types of bogies are assembled one by one along a mixed-model assembly line. The above manufacturing process of bogies can be extracted as the MAJSP-LS.

To demonstrate the influence of LS on the MAJSP, an illustrative case is provided and the scenarios with or without LS are compared. This case

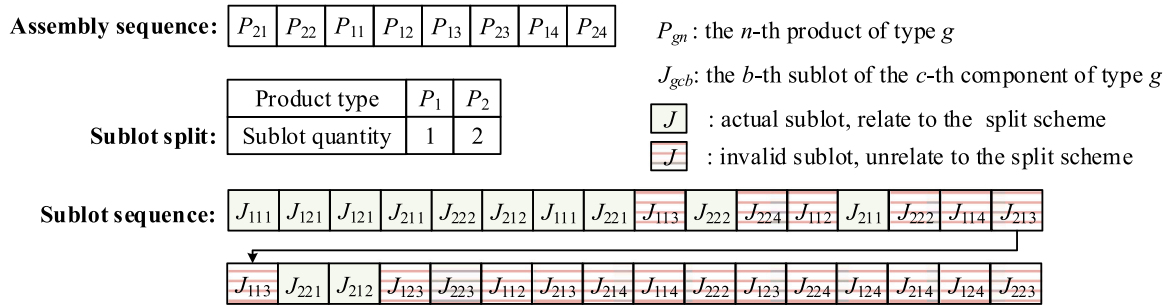


Fig. 4. Solution representation.

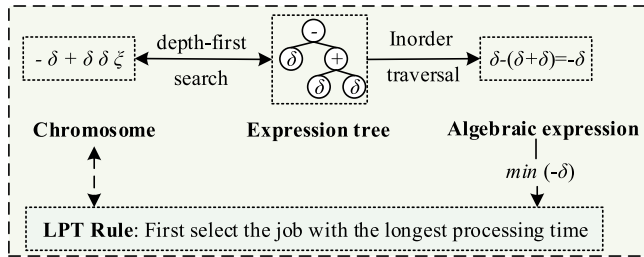


Fig. 5. The transformation process from a chromosome to a specific dispatching rule.

involves two types of products, and the lot size of each product type is four. For each product, it consists of two components with each of them having two processing operations. In sum, a total of $2 * 4 * 2 * 2 = 32$ processing operations of 16 components are first performed on three

machines, and sequence-dependent setups are considered. Then, these processed components are assembled along a mixed-model assembly line with two stations to form $2 * 4 = 8$ products.

The scenario of the MAJSP without LS: In the processing stage, components of a lot are processed continuously and transferred to the next operation together after they are all processed. In the assembly stage, eight products are assembled one by one in a mixed-model assembly line. Fig. 1 shows the Gantt chart of this scenario. In Fig. 1, the total assembly completion time is 3907 min, and the total setup time is 144 min.

The scenario of MAJSP with LS: The equal and consistent sublots with intermingling is considered. Each type of product is split into two equal sublots with each sublot having two components. The processing and transferring unit is sublot, and sublots from different lots can be processed with intermingling. The assembly stage is the same as that of the scenario of MAJSP without LS. Fig. 2 shows the Gantt chart of this scenario. In Fig. 2, the total assembly completion time and setup time are respectively 3576 and 174 min.

In sum, the scenario of MAJSP without LS requires fewer setups. However, the flow of production in the processing stage is slow, which leads to a serious delay in subsequent assembly. For the scenario of MAJSP with LS, setup time is slightly increased. But the accelerated flow of production allows components to complete earlier, and then these completed components can be assembled as earlier as possible. As a result, the assembly completion time is greatly shortened.

4. Model formulation

The studied problem is described below. Considering that P types of products need to be manufactured. Each product type $p \in P$ has a single-level assembly structure and is assembled from $|C_p|$ components with each component $c \in |C_p|$ having $|O_{cp}|$ processing operations. The demand for product type p is l_p , and accordingly each type of component $c \in |C_p|$ has l_p identical items as a lot. In the processing stage, all components are processed on M machines according to their specific processing routes. Lot streaming is considered to accelerate the flow of production. Each lot is divided into several equal and consistent sublots. G_{pd} is the sublot size of the d -th sublot of product type p . Besides, sublots from different lots are allowed to be processed with intermingling. Sequence-dependent setup time s_{pcpc} exists between two successive sublots from different lots. Once all the processing operations of a component are finished, the component is transported to the specified station for assembly. In the assembly stage, components of each product are assigned to W stations of an assembly line and then assembled with single-piece flow to form final products. The assembly time of product type p on station w is β_{pw} .

The main task of the studied problem is to minimize the manufacturing and setup costs by optimizing the sublot split, sublot sequence and assembly sequence of products.

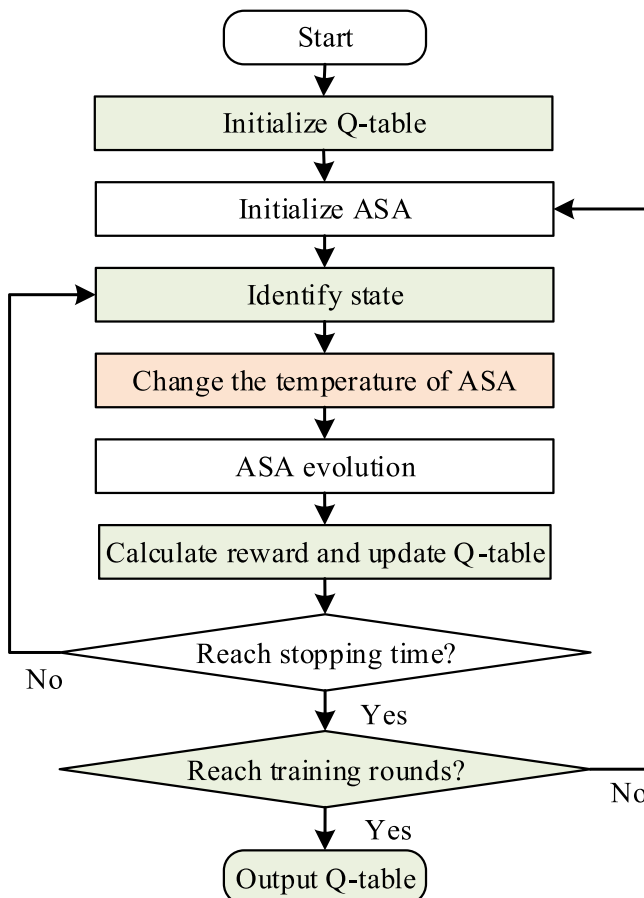


Fig. 6. Training procedure of Q-learning for temperature control.

4.1. Assumptions and notations

The MAJSP-LS model is formulated based on the following assumptions:

- (1) The equal and consistent sublots with intermingling is adopted in the processing stage.
- (2) Sequence-dependent setup times are considered.
- (3) The transportation time of components is ignored.
- (4) At most one machining/assembly operation is performed at a time.
- (5) Each machining/assembly operation is executed exactly once, and preemption is not allowed.
- (6) Assembly station assignment of components is known in advance.
- (7) Assembling multiple components of a product at one station is taken as one assembly operation.

All notations used for the MAJSP-LS model are listed below.

Indexes:

p	The index of product type, $p = 1, 2, 3, \dots, P $.
c	Component index, $c = 1, 2, 3, \dots, C_p $.
o	The index of processing operations, $o = 1, 2, 3, \dots, O_{cp} $.
d	Sublot index, $d = 1, 2, 3, \dots, D $.
n	The index of the component within a sublot, $n = 1, 2, 3, \dots, N $.
m	Machine index, $m = 1, 2, 3, \dots, M $.
w	Station index, $w = 1, 2, 3, \dots, W $.
k	The position index on machine m , $k = 1, 2, 3, \dots, K_m $, where $ K_m = \sum_{p=1}^{ P } \sum_{d=1}^{ D } \sum_{c=1}^{ C_p } \sum_{o=1}^{ O_{cp} } D_{pdco} \cdot D_{pdcom}$ takes 1 if the operation j_{pdco} is performed on machine m ; otherwise, 0.
h	The position index on each station, $h = 1, 2, 3, \dots, H $, where $ H = \sum_{p=1}^{ P } l_p$.
j_{pdco}	The o -th operation of the d -th sublot of component c of product type p .
e_{pdn}	The n -th product of the d -th sublot of product type p .

Parameters:

l_p	Lot size of product type p .
$s_{pcp'c'}$	Setup time between component c of product type p and component c' of product type p' . If $p = p'$ and $c = c'$, $s_{pcp'c'} = 0$.
s_{pc}^0	Setup time for a machine to start processing component c of product type p .
b_{cpw}	If component c of product type p is assigned to station w , $b_{cpw} = 1$; else, $b_{cpw} = 0$.
β_{pw}	Assembly time of product type p on station w .
δ_{mopc}	Processing time. If the o -th operation of component c of product type p is performed on machine m , δ_{mopc} is a positive number; otherwise, $\delta_{mopc} = 0$.
θ_1	The coefficient of manufacturing costs.
θ_2	The coefficient of setup costs.
φ	A big positive number.

Variables:

X_{mkpdco}	0–1 variable. X_{mkpdco} takes 1, if j_{pdco} is performed at the k -th position of machine m .
Y_{pdnh}	0–1 variable. Y_{pdnh} takes 1, if e_{pdn} is assembled at the h -th position.
Z_{pd}	0–1 variable. Z_{pd} takes 1, if product type p exists sublots d .
Ψ_{pdn}	0–1 variable. Ψ_{pdn} takes 1, if the d -th sublot of product type p exists the n -th product.
G_{pd}	Positive variable. Sublot size of the d -th sublot of product type p .
Λ_{pdco}^s	Continuous variable. The processing start time of operation j_{pdco} .
Λ_{pdnc}^e	Continuous variable. The processing finish time of component c of e_{pdn} .
B_{pdnw}^e	Continuous variable. The assembly finish time of e_{pdn} on station w .
V_{mk}^s	Continuous variable. The start time of the k -th position of machine m .
A_{wh}^s	Continuous variable. The start time of the h -th position of station w .
A_{wh}^e	Continuous variable. The end time of the h -th position of station w .

4.2. Objectives and constraints

As defined in Eq.(1), the objective is to minimize manufacturing and setup costs. Among them, manufacturing costs depend on the total assembly completion time calculated by Eq.(2). Setup costs are influenced by the total setup time calculated by Eq.(3).

$$\text{minobj} = f_1 \bullet \theta_1 + f_2 \bullet \theta_2 \quad (1)$$

$$f_1 \geq \sum_p \sum_d \sum_n B_{pdn,|w|}^e \quad (2)$$

$$f_2 \geq \sum_m \sum_k \sum_p \sum_c \sum_d \sum_o \sum_p \sum_c \sum_d \sum_o X_{mkpdco} \bullet X_{m,k+1,p'c'd'o} \bullet s_{pcp'c'} + \sum_m \sum_p \sum_c \sum_d \sum_o X_{m,1,pdco} \bullet s_{pc}^0 \quad (3)$$

To achieve the above objectives, the following five types of constraints need to be satisfied.

Sublot split constraints: Eq.(4) ensures that the ahead sublot has a higher priority to accommodate components. Eqs.(5–6) achieve that if a sublot does not exist, its sublot size is zero; else, its sublot size is a positive number. Eq.(7) denotes that for each type of product, the total sublot size is equal to the lot size. Eqs.(8–9) jointly achieve the equal and consistent sublots, and if a lot cannot be equalized, the sublot size of the first sublot is set to be smaller than that of other sublots.

$$Z_{pd} \geq Z_{p,d+1}, \forall p, d < |D| \quad (4)$$

$$Z_{pd} \geq G_{pd} / \varphi, \forall p, d \quad (5)$$

$$Z_{pd} \leq G_{pd}, \forall p, d \quad (6)$$

$$\sum_d G_{pd} = l_p, \forall p \quad (7)$$

$$G_{pd} \leq G_{p,d+1} + \varphi \bullet (2 - Z_{pd} - Z_{p,d+1}), \forall p, d < |D| \quad (8)$$

$$G_{pd} \geq G_{p,d+1} - \varphi \bullet (2 - Z_{pd} - Z_{p,d+1}), \forall p, d > 1 \text{ and } d < |D| \quad (9)$$

Sublot sequence constraints: Eq.(10) limits that each processing operation is performed exactly once. Eq.(11) denotes that each event position of machines executes at most one processing operation. Eq.(12) means that for each machine, the ahead event position has a higher processing priority. Eq.(13) demands that only when all components within a sublot are finished, can the next operation of the sublot start. Eqs.(14–15) associate the start time of sublots with that of the corresponding event positions of machines.

$$\sum_m \sum_k X_{mkpdco} = Z_{pd}, \forall p, d, c, o \quad (10)$$

$$\sum_p \sum_d \sum_c \sum_o X_{mkpdco} \leq 1, \forall m, k \quad (11)$$

$$\sum_p \sum_d \sum_c \sum_o X_{mkpdco} \geq \sum_p \sum_d \sum_c \sum_o X_{m,k+1,pdco}, \forall m, k < |K_m| \quad (12)$$

$$\Lambda_{pdco}^s + \sum_{m|\delta_{mopc}>0} \delta_{mopc} \bullet G_{pd} \leq \Lambda_{pdco+1}^s + \varphi \bullet (1 - Z_{pd}), \forall p, d, c, o < |O_{cp}| \quad (13)$$

$$\Lambda_{pdco}^s \leq V_{mk}^s + \varphi \bullet (1 - X_{mkpdco}), \forall m, k, p, d, c, o \quad (14)$$

$$\Lambda_{pdco}^s \geq V_{mk}^s - \varphi \bullet (1 - X_{mkpdco}), \forall m, k, p, d, c, o \quad (15)$$

Sequence-dependent setup constraints: Eq.(16) denotes that the sublot assigned to the first event position can start only after the corresponding machine tool is ready. Eq.(17) means that on each machine, the start time of the next sublot is larger than the finish time of its previous sublot plus the corresponding setup time.

$$V_{mk}^s \geq \sum_p \sum_d \sum_c \sum_o X_{mkpdco} \bullet s_{pc}^0, \forall m, k = 1 \quad (16)$$

$$V_{mk}^s + \sum_p \sum_d \sum_c \sum_o \sum_p \sum_d \sum_c \sum_o s_{pcp'c'} \bullet X_{mkpdco} \bullet X_{m,k+1,p'd'c'o} + \sum_p \sum_d \sum_c \sum_o \delta_{mopc} \bullet G_{pd} \bullet X_{mkpdco} \leq V_{m,k+1}^s, \forall m, k < |K| \quad (17)$$

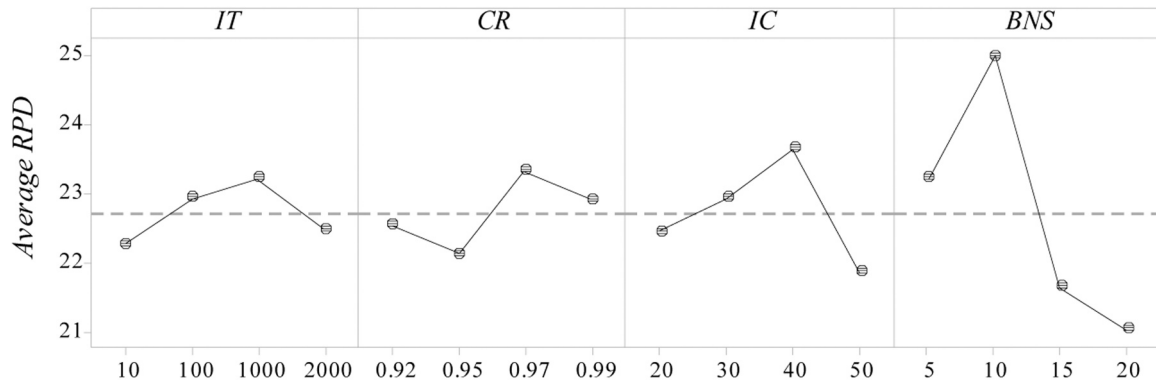


Fig. 7. The main effect plot for SNR.

Table 1

Parameter configurations of the variants of ASA.

Parameter	Parameter level	SA	SA-O	SA-S	SA-OS	ASA
IT	{10, 100, 1000, 2000}	1000	1000	1000	1000	1000
CR	{0.92, 0.95, 0.97, 0.99}	0.99	0.99	0.95	0.97	–
IC	{20, 30, 40, 50}	30	30	50	40	40
BNS	{5, 10, 15, 20}	–	–	5	10	10
NS	{5, 10, 15, 20}	5	5	–	–	–

Table 2

Parameter configurations of other algorithms.

Algorithm	Parameter	Parameter level	value
TS	Tabu tenure	{3, 5, 8, 10}	8
	Neighborhood size	{5, 10, 15, 20}	15
EICA	Population size	{100, 200, 300, 400}	200
	Number of empires	{10, 15, 20, 25}	20
	Possibility of assimilation	{0.92, 0.94, 0.96, 0.98}	0.92
GA-DR	Possibility of revolution	{0.1, 0.15, 0.17, 0.2}	0.2
	Population size	{50, 100, 200, 300}	50
	Crossover rate	{0.92, 0.94, 0.96, 0.98}	0.98
IABC	Mutation rate	{0.1, 0.15, 0.17, 0.2}	0.15
	Population size	{30, 50, 70, 90}	30
	Restart limit	{20, 30, 40, 50}	30

Sequence constraints of components within a subplot: A binary variable Ψ_{pdn} is introduced to index the component within a subplot. Based on the processing start time of each subplot and the index of components within a subplot, the processing finish time of each component can be determined by Eqs.(18–19). In addition, Ψ_{pdn} obeys constraints (20–21). Among them, Eq.(20) determines the order of components in each subplot. Eq.(21) ensures that the total number of components within a subplot is equal to the subplot size.

$$\Lambda_{pdnc}^e \leq \Lambda_{pdco}^s + n \bullet \sum_{m|\delta_{mopc}>0} \delta_{mopc} + \varphi \bullet (1 - \Psi_{pdn}), \forall p, d, c, n, o = |O_{cp}| \quad (18)$$

$$\Lambda_{pdnc}^e \geq \Lambda_{pdco}^s + n \bullet \sum_{m|\delta_{mopc}>0} \delta_{mopc} - \varphi \bullet (1 - \Psi_{pdn}), \forall p, d, c, n, o = |O_{cp}| \quad (19)$$

$$\Psi_{pdn} \geq \Psi_{pd,n+1}, \forall p, d, n < |N| \quad (20)$$

$$\sum_n \Psi_{pdn} = G_{pd}, \forall p, d \quad (21)$$

Mixed-model assembly constraints: Similar to Eqs.(10–11), Eqs. (22–23) constrain the execution of each assembly operation. Eq.(24) demands that for a given component, its assembly start time must be bigger than its processing finish time. Eq.(25) gives the relationship between the start time and end time of each assembly station. Eq.(26) gives the start and end time relation of two assembly operations assigned to successive stations. Eq.(27) establishes the start and end time relation of two successive event positions of a station. Eqs.(28–29) associate the assembly finish time of products with the end time of the corresponding event positions of stations.

$$\sum_h Y_{pdnh} = \Psi_{pdn}, \forall p, d, n \quad (22)$$

$$\sum_p \sum_d \sum_n Y_{pdnh} \leq 1, \forall h \quad (23)$$

$$\Lambda_{pdnc}^e \leq A_{wh}^s + \varphi \bullet (1 - Y_{pdnh}), \forall p, d, c, n, w, h, b_{cpw} = 1 \quad (24)$$

$$A_{wh}^s + \sum_p \sum_d \sum_n \beta_{pw} \bullet Y_{pdnh} \leq A_{wh}^e, \forall w, h \quad (25)$$

$$A_{wh}^e \leq A_{w+1,h}^s, \forall w < |W|, h \quad (26)$$

$$A_{wh}^e \leq A_{w,h+1}^s, \forall w, h < |H| \quad (27)$$

$$B_{pdnw}^e \leq A_{wh}^e + \varphi \bullet (1 - Y_{pdnh}), \forall p, d, n, w, h \quad (28)$$

$$B_{pdnw}^e \geq A_{wh}^e - \varphi \bullet (1 - Y_{pdnh}), \forall p, d, n, w, h \quad (29)$$

Table 3

The learned Q-table for temperature control.

Evolutionstate	Action					Evolutionstate	Action				
	a1	a2	a3	a4	a5		a1	a2	a3	a4	a5
(Cr ₁ , Fu ₁)	6.22	72.12	3.90	440.14	14.22	(Cr ₂ , Fu ₄)	1.21	0	19.50	0.01	0
(Cr ₁ , Fu ₂)	24.17	207.97	12.26	10.18	26.05	(Cr ₂ , Fu ₅)	0.66	2.36	2.16	40.07	0.10
(Cr ₁ , Fu ₃)	0	0.32	0.74	0.57	69.22	(Cr ₃ , Fu ₁)	10.44	88.86	0.76	0.08	5.16
(Cr ₁ , Fu ₄)	0	0	9.45	22.95	0	(Cr ₃ , Fu ₂)	1.09	0.01	54.56	2.16	3.58
(Cr ₁ , Fu ₅)	0.26	0.04	0.04	0.36	18.62	(Cr ₃ , Fu ₃)	8.42	32.55	1.59	3.65	2.66
(Cr ₂ , Fu ₁)	0.40	0	0.38	42.87	2.17	(Cr ₃ , Fu ₄)	2.48	2.19	20.75	0.88	1.00
(Cr ₂ , Fu ₂)	7.24	1.28	79.94	0.59	0.44	(Cr ₃ , Fu ₅)	4.58	71.52	3.84	8.08	15.37
(Cr ₂ , Fu ₃)	38.44	1.12	0.31	0.02	0						

Table 4
Comparison results of six rules.

Case	Random		SPT		LPT		TWK		MTWK		Proposed	
	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>
B1	3877724	3472335	4275106	3473415	4718084	4251717	4574367	4326498	2821194	2395179	2632900	2350053
B2	1353534	1124843	1705606	1483069	1529425	1351051	1577456	1511770	924982	810627	886331	867413
B3	1133532	918830	1130525	824769	1411876	1174965	1655599	1416910	826396	765545	784272	760166
B4	1136721	937191	1167625	962895	1103970	943584	1567052	1292736	833548	722253	770480	655812
B5	2863900	2438338	3098107	2775498	3198393	2952910	3366439	3187281	2042913	1620166	1793191	1589467
B6	1394947	1199609	1984171	1805739	1673573	1402844	1707416	1647883	1010854	913394	909957	879276
B7	1871521	1572918	1802439	1607522	2268917	1864786	2130503	1897100	1451248	1247940	1427448	1340948
B8	1624778	1386752	1479858	1348664	2120689	1855712	1909046	1481485	1195284	1050638	1140516	987499
B9	2257758	1835718	2386791	2199474	2275270	1860492	2746323	2574792	1640470	1500783	1536258	1420740
B10	1170300	899388	1192319	951966	1440682	1241854	1586141	1320651	859981	809262	769593	712117
B11	1452314	1161411	1332764	1082575	1529878	1360075	1407847	1274287	1067607	931914	998503	948242
B12	767550	594708	1096289	837142	1064291	949066	847492	722514	524888	468828	490174	448610
B13	1527503	1286881	1611041	1348525	1531749	1268660	1776802	1679517	1215243	1039297	1148251	1092355
B14	1374338	1129562	1657217	1293614	1349423	1203790	1846763	1575812	1016257	955794	921488	849114
B15	2128537	1659951	2033613	1508461	2583225	2269762	2236723	1972110	1628685	1487333	1330353	1170006
B16	1311438	1122188	1274868	915473	1525332	1292655	1524208	1394913	911601	790412	879733	765635
B17	5417464	4630540	6348823	5353400	6833477	6122340	6268209	5902660	4027835	3499300	3795139	3414880
B18	5060138	4462332	7337931	6439613	6247158	5415883	6129282	5884406	3525651	3155372	3238021	3148996
B19	1441105	1234882	1502367	1355934	1794987	1568580	1910663	1623276	1045036	919392	1011131	968824
B20	1907510	1684808	1905527	1546692	2430309	2159240	2287465	1977412	1313869	1139304	1260388	1112136
B21	3640298	3007106	4022906	3455302	3668280	3066208	4195921	3946595	2621393	2349367	2452242	2374482
B22	2190273	1967122	2761448	2288790	2478328	2226214	3254343	2622910	1630311	1527672	1541625	1376092
B23	2640356	2264780	2811209	2291535	2869588	2615612	2797729	2447794	2016470	1807463	1820670	1732494
B24	2815350	2450577	3812580	2882899	4272176	3612845	3303845	3071192	1915146	1713046	1910366	1737287
B25	5696489	4822998	6686585	5428769	7270208	5967742	6291503	5917788	4215748	3752130	4206666	3723999
B26	8008765	7041732	9591537	8408812	9145720	7869820	9012224	8534845	5692627	5116636	5555543	5300026
B27	3466607	3048108	4728641	4249560	4360135	3867656	4209096	4055684	2414217	2143998	2214494	2111076
B28	3325519	2985576	4392389	3925153	4643583	3906552	4552285	4062560	2341557	2181576	2322984	2161335
B29	3319249	2893978	3757215	3155436	4906849	4150534	4640336	4072658	2312300	2075216	2161723	2014624
B30	3548709	3011137	3617506	3135238	4113144	3670730	3861344	3632139	2517442	2285751	2461436	2278506
B31	4054635	3533269	4184272	3663532	5012126	4647882	5005455	4478163	2759585	2644850	2729542	2530057
B32	2788197	2379505	4126092	3149400	3804147	3457650	3623678	3174235	2023029	1753265	1948500	1742625

Table 5

Comparison results of algorithms with or without adaptive neighborhood search.

Case	SA		SA-O		SA-S		SA-OS	
	Avgfitness	Minfitness	Avgfitness	Minfitness	Avgfitness	Minfitness	Avgfitness	Minfitness
B1	2159995	2100528	2138119	2044944	2132609	2051973	2131404	2054772
B2	799350	729726	784514	704466	788866	726080	773587	706340
B3	675493	610005	666205	589336	657455	585662	668488	612722
B4	625734	589638	621461	572385	608010	570549	605325	559329
B5	1542617	1476778	1538684	1440071	1519802	1440794	1508223	1430779
B6	820712	777855	821611	767903	804310	757972	808410	762536
B7	1078963	961544	1067749	962794	1031989	970108	1044993	972878
B8	924365	887539	921680	868197	916925	875290	913170	871689
B9	1337542	1289982	1329946	1256274	1306252	1264725	1303152	1248795
B10	692283	647534	699854	669172	687844	654951	685373	624046
B11	780357	712286	779962	721539	791202	723217	767504	704588
B12	437440	411294	436825	418636	432770	412956	433732	426466
B13	880761	802214	899534	816559	880850	810675	870088	797995
B14	839805	760102	838042	748034	832879	723394	834985	738304
B15	1146034	1106952	1145630	1108641	1135145	1078718	1128405	1067524
B16	760625	736779	764882	717068	761495	733074	753544	722011
B17	3038407	2839660	2954399	2728360	2933541	2754780	2872485	2747340
B18	2956880	2748012	2918454	2717779	2901327	2771972	2891137	2674680
B19	803876	731620	801844	729036	791132	737812	790498	722178
B20	1037077	973984	1033447	973264	1023149	943256	1016422	964028
B21	2127713	1962121	2087037	1873164	2096772	1942360	2050131	1883554
B22	1297859	1249222	1292399	1199644	1269579	1186228	1272464	1179118
B23	1489332	1360389	1520852	1396583	1460206	1333146	1486268	1292737
B24	1634723	1540155	1625174	1570705	1601817	1473124	1595921	1438431
B25	3209089	3077603	3162706	2893996	3165736	2913080	3031938	2824249
B26	4768406	4506638	4753433	4297918	4683914	4333583	4611679	4199003
B27	2006134	1908460	1967262	1841198	1979841	1886196	1937073	1777094
B28	1961767	1792173	1943818	1840042	1898441	1772921	1904757	1814847
B29	1774028	1673180	1736459	1628714	1721050	1604962	1699178	1608954
B30	2041915	1874898	2013184	1830562	1961382	1761606	1928954	1709796
B31	2302272	2172869	2265353	2135770	2249211	2094728	2249925	2068997
B32	1654858	1602405	1642070	1595595	1632122	1578530	1628621	1549405

4.3. Model linearization and complexity analysis

The consideration of LS and sequence-dependent setup leads to two types of nonlinear terms. The first term is the positive variable times the binary variable, $G_{pd} \bullet X_{mkpdc}$. The second term is the multiplication of two binary variables, $X_{mkpdc} \bullet X_{m,k+1,p'd'c'o'}$. To eliminate non-linearity, intermediate variables and some indispensable constraints are introduced below.

To eliminate $G_{pd} \bullet X_{mkpdc}$, an intermediate variable I_{mkpdc}^{XG} is introduced, and Eqs.(30–33) are added to ensure that when X_{mkpdc} takes 0, I_{mkpdc}^{XG} takes 0; when X_{mkpdc} takes 1, I_{mkpdc}^{XG} is equal to G_{pd} .

$$I_{mkpdc}^{XG} \leq \varphi \bullet X_{mkpdc}, \forall m, k, p, d, c, o \quad (30)$$

$$I_{mkpdc}^{XG} \geq -\varphi \bullet X_{mkpdc}, \forall m, k, p, d, c, o \quad (31)$$

$$I_{mkpdc}^{XG} \leq G_{pd} + \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (32)$$

$$I_{mkpdc}^{XG} \geq G_{pd} - \varphi \bullet (1 - X_{mkpdc}), \forall m, k, p, d, c, o \quad (33)$$

To remove $X_{mkpdc} \bullet X_{m,k+1,p'd'c'o'}$, a binary variable $I_{mkpdcop'd'c'o'}^{XX}$ is introduced and Eqs.(34–36) are added to ensure that if both X_{mkpdc} and $X_{m,k+1,p'd'c'o'}$ take 1, $I_{mkpdcop'd'c'o'}^{XX}$ takes 1; else, $I_{mkpdcop'd'c'o'}^{XX}$ takes 0.

$$I_{mkpdcop'd'c'o'}^{XX} \leq X_{mkpdc}, \forall m, k, p, d, c, o, p', d', c', o' \quad (34)$$

$$I_{mkpdcop'd'c'o'}^{XX} \leq X_{m,k+1,p'd'c'o'}, \forall m, k, p, d, c, o, p', d', c', o' \quad (35)$$

$$I_{mkpdcop'd'c'o'}^{XX} \geq X_{mkpdc} + X_{m,k+1,p'd'c'o'} - 1, \forall m, k, p, d, c, o, p', d', c', o' \quad (36)$$

Finally, by using I_{mkpdc}^{XG} and $I_{mkpdcop'd'c'o'}^{XX}$ to respectively replace $G_{pd} \bullet X_{mkpdc}$ and $X_{mkpdc} \bullet X_{m,k+1,p'd'c'o'}$, a MILP model is formulated.

The MILP model contains four binary variables $\{X_{mkpdc}, Y_{pdnh}, Z_{pd}, \Psi_{pdn}\}$ and one positive variable G_{pd} . Based on the provided notations, the maximum number of event positions on each station and machine m is $\sum_{p=1}^{|P|} l_p$ and $|K_m| = \sum_{p=1}^{|P|} \sum_{d=1}^{|D|} \sum_{c=1}^{|C_p|} \sum_{o=1}^{|O_p|} D_{pdcom}$ respectively, and both the maximum number of sublots and the maximum number of components within a subplot are l_p . Thus, the size of variable space of $\{X_{mkpdc}, Y_{pdnh}, Z_{pd}, \Psi_{pdn}, G_{pd}\}$ is $\sum_{m=1}^{|M|} (K_m)^2, \sum_{p=1}^{|P|} (l_p)^3, \sum_{p=1}^{|P|} l_p, \sum_{p=1}^{|P|} (l_p)^2$ and $\sum_{p=1}^{|P|} l_p$ respectively, and the total size is $\sum_{m=1}^{|M|} (K_m)^2 + \sum_{p=1}^{|P|} (l_p)^3 + \sum_{p=1}^{|P|} (l_p)^2 + 2 \sum_{p=1}^{|P|} l_p$. Besides, the model contains constraints (4–36). The maximum number of constraints is $3 \sum_{m=1}^{|M|} (K_m)^3 + 9 \sum_{m=1}^{|M|} (K_m)^2 + |W| \bullet \sum_{p=1}^{|P|} ((l_p)^3 \bullet |Cp|) + 2|W| \bullet \sum_{p=1}^{|P|} (l_p)^3 + 2 \sum_{p=1}^{|P|} ((l_p)^2 \bullet |Cp|) + 2 \sum_{p=1}^{|P|} (l_p)^2 + 5 \sum_{p=1}^{|P|} l_p + 3|W| \bullet l_p + |W| + |P| + |M| - 2|K_m| - 2$. It can be seen that the problem size is very sensitive to the total number of operations on each machine $|K_m|$ and the lot size of each type of products l_p . As the size of problems grows, the size of decision variables and constraints increases exponentially.

5. Adaptive simulated annealing algorithm

Simulated annealing (SA) is a local search metaheuristic inspired by the annealing of metals. Due to its powerful local search ability and easy implementation, SA has been successfully applied to solve a wide range of production scheduling problems including the JSP [44], MAJSP [3], three-machine assembly scheduling [45], assembly flow shop scheduling with LS [46] and so on. More details about the SA can refer to the literature [47].

In this work, an adaptive simulated annealing algorithm (ASA) is developed to solve the studied problem. As shown in Fig. 3, the adaptivity of the ASA is enhanced from the following four aspects. (1) A subplot sequence rule considering problem-specific attributes is dug out

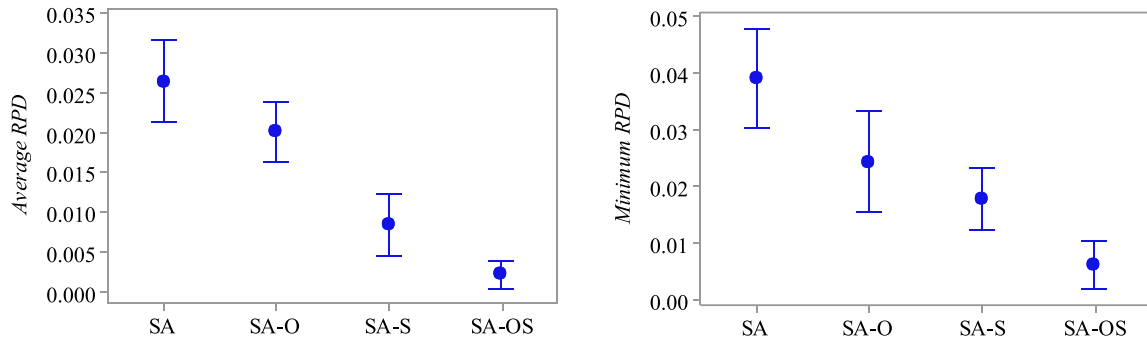


Fig. 8. The interval plot of SA, SA-O, SA-S and SA-OS on average and minimum RPD.

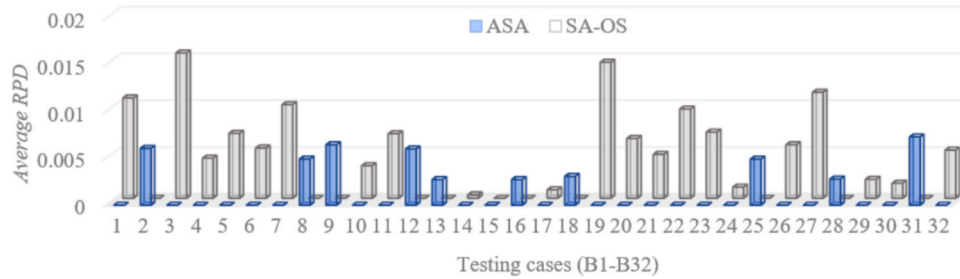


Fig. 9. The comparison results of ASA and SA-OS on average RPD metric.

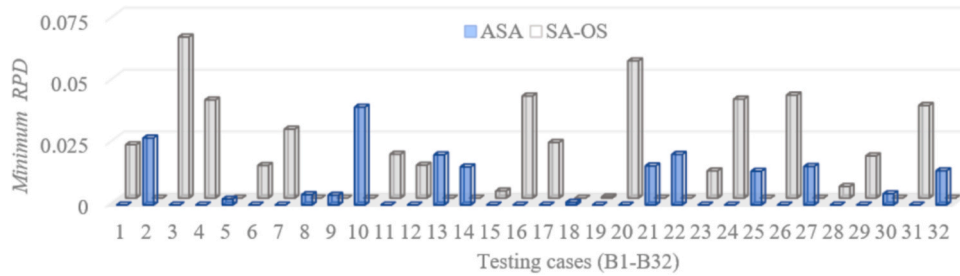


Fig. 10. The comparison results of ASA and SA-OS on minimum RPD metric.

to quickly locate the potential solution space. (2) A temperature adaptive control mechanism is developed to avoid being trapped in local optima. (3) An adaptive selection mechanism of neighborhood operators is designed to increase the chance of finding potential solutions. (4) An adaptive adjustment mechanism of neighborhood size is proposed to balance computing resources and the coverage of neighborhood space.

5.1. Encoding

For the studied problem, each solution includes three parts: subplot split, subplot sequence and assembly sequence.

Since the equal and consistent sublots method is adopted, the subplot split only needs to determine the subplot quantity. Considering that an instance contains P types of products, and the lot size of each type of product is l_p , this segment can be represented by P elements. Each element denotes the subplot quantity of the corresponding product type, and each element takes a value from the interval $[1, l_p]$.

The subplot sequence segment determines the processing priority of sublots. For different subplot split schemes, the sublots that need to be sequenced change accordingly. To adapt to any subplot split scheme, the processing sequence of the maximum sublots is provided in the encoding. But, only those sublots corresponding to the given subplot split scheme are focused in the decoding phase, and these sublots are called actual sublots. Considering that each product consists of C_p components,

and each component has $|O_{cp}|$ operations, the subplot sequence segment is encoded with $\sum_{p=1}^P \sum_{c=1}^{C_p} l_p \cdot |O_{cp}|$ genes. Each gene is represented by a subplot number and the index of the gene denotes the processing priority. Note that, all the operations of a subplot are represented by the same subplot number, and a subplot number occurs k times to represent its k -th operation.

The assembly sequence determines the assembly priority of products. This segment is encoded with $\sum_{p=1}^P l_p$ genes. Each gene represents a product, and its index denotes the assembly priority of the product.

Based on the above encoding, a solution of the case in Section 3 can be encoded as Fig. 4. It can be seen from Fig. 4 that the first product of type two is assembled first, and followed by the second one of type two. The two types of products are respectively divided into one and two sublots. Besides, the maximum subplot of two types of product is four, and each type of product has two components with two processing operations. Thus, the maximum length of the subplot sequence segment is $2 * 4 * 2 * 2 = 32$. But, the valid sublots corresponding to the given subplot split scheme are $1 * 2 * 2 + 2 * 2 * 2 = 12$, and their processing sequence is $\{J_{111}, J_{121}, J_{121}, J_{211}, J_{222}, J_{212}, J_{111}, J_{221}, J_{222}, J_{211}, J_{221}, J_{212}\}$.

Table 6
Comparison results of the ASA with other algorithms.

Case	IABC		GA-DR		TAVNS		TS		EICA		ASA	
	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>	<i>Avgfitness</i>	<i>Minfitness</i>
B1	2751586	2581794	2348273	2347461	2443795	2198988	2397675	2266047	2514008	2250666	2143778	2065365
B2	871143	797593	848884	848762	830876	749108	808629	764273	810980	755076	777074	706081
B3	781960	734756	757880	752686	728906	652960	700687	651453	723531	670978	669424	586982
B4	763978	664875	649868	649848	707268	614526	663788	611592	674458	611787	615870	576606
B5	1898414	1796075	1699102	1695712	1679362	1554043	1655542	1503341	1690165	1552644	1533296	1451857
B6	932604	858945	872145	872138	863212	819524	852748	800657	855917	812866	822118	786456
B7	1315060	1244672	1396831	1396198	1221207	1087360	1155963	1058872	1146923	1073470	1051177	971944
B8	1151882	1083143	986027	985982	1031201	939043	1001890	908820	1013150	936107	920529	881591
B9	1496770	1406097	1432506	1431852	1378387	1242417	1346988	1269933	1369427	1275138	1325610	1256367
B10	785817	746125	699529	698635	746452	681137	705931	650041	725024	650072	695031	652404
B11	957237	876453	935922	935922	901180	800748	834159	792491	862179	820090	796650	715859
B12	498911	460222	445540	445540	475726	424196	446890	416278	453663	423626	437454	423240
B13	1021790	904141	1081322	1081315	960434	907003	924282	869201	930303	855752	868735	797496
B14	938046	894138	847759	846342	886670	783972	867870	768446	869280	787444	840714	764204
B15	1429190	1339828	1147046	1144546	1340763	1174693	1239356	1169155	1293102	1208471	1145743	1085949
B16	898199	845762	784232	783635	844926	771392	792528	733145	813906	761018	759966	722068
B17	3866226	3571080	3418214	3410140	3420556	3068220	3334240	3093700	3454067	3325860	2932295	2733720
B18	3553230	3314768	3144284	3138917	3163401	2967943	3122408	2868518	3227083	3030899	2933824	2699740
B19	980326	914784	981572	981518	868131	807760	849155	779538	854734	789854	782095	724250
B20	1283852	1197708	1079806	1079164	1192317	1083976	1104359	996684	1150359	1027200	1024766	962860
B21	2640416	2508846	2330235	2327360	2318926	2132762	2270296	2121628	2337911	2211514	2091391	1922960
B22	1590257	1532596	1346122	1337082	1459835	1215304	1391645	1272004	1442408	1350360	1284231	1208770
B23	1926708	1786159	1733063	1731704	1690620	1528800	1600468	1454140	1701626	1594066	1490783	1346935
B24	1987017	1881978	1720292	1717767	1795313	1614933	1673948	1568261	1772505	1648843	1606502	1500443
B25	4092755	3939231	3595579	3591500	3547789	3294676	3471971	3262595	3632738	3496992	3119709	2824887
B26	5901808	5528333	5236392	5228044	5193426	4748580	5038062	4669609	5316099	4888465	4718477	4385844
B27	2475211	2375066	2118110	2113596	2170169	1981312	2124493	1899990	2226643	2143306	1986740	1865218
B28	2344144	2255857	2115423	2103517	2153717	1936465	2059491	1889097	2155750	2010364	1936882	1796459
B29	2376087	2225200	2022923	2021100	2086871	1909798	1918270	1796590	2089222	1879926	1718088	1633628
B30	2613868	2504127	2163860	2151437	2318372	1992112	2176102	1993901	2304776	2187992	1991928	1836533
B31	2906516	2773456	2527438	2510749	2546854	2305358	2438379	2282958	2587550	2413374	2248872	2133514
B32	2152410	1990250	1697222	1690745	1860154	1648645	1737888	1604840	1918440	1796690	1648081	1594560

5.2. Initialization and actual subplot-oriented decoding

Each solution in the encoding consists of three segments. The initialization of the three segments is described below. For the assembly sequence segment, all types of products are first ranked randomly. Then, all products belonging to the same type are sequenced together, and the product with a smaller index is sequenced first. The subplot split segment is generated by randomly taking values from the interval $[1, l_p]$. As for the subplot sequence segment, a subplot sequence rule is dug out in Section 5.3 and used to create a sequence.

Considering that the subplot sequence created by the extracted rule is closely related to the assembly sequence, multiple candidate solutions corresponding to different assembly sequences are first created, and then the solution with the best fitness is selected as the final initial solution.

For each solution, the decoding procedure is described below:

Step 1: Set the available time of machines, stations and sublots to zero.

Step 2: Find out all actual sublots. Based on the equal and consistent sublots, determine the subplot size and processing time of each actual subplot.

Step 3: Take one subplot from the subplot sequence segment. If the subplot is an invalid subplot, continue to take the next one; otherwise, calculate the setup time and determine the start and end time of the subplot under the premise of satisfying machine available, subplot available and setup time constraints, and then update the available time of machines and sublots. Repeat step 3 until the start and end times of all the actual sublots are obtained.

Step 4: According to the start time of sublots, and the index of components within a subplot, derive the processing finish time of each component.

Step 5: Take one product from the assembly sequence segment. Subsequently, find out the finish time of all components of the product, and check the assigned station of each component. Then, determine the assembly start and end times of each product on each station under the premise of satisfying station available and component available constraints, and then update the available time of stations. Repeat step 5 until the finish times of all products are obtained.

Step 6: Calculate the manufacturing and setup costs according to the assembly completion time and setup time.

5.3. GEP-based subplot sequence rule mining

Gene expression programming (GEP) is a genotype/phenotype evolutionary algorithm. Due to the powerful data mining and self-study ability, GEP has been widely adopted to dig out problem-specific rules [48].

In GEP, each chromosome is a fixed-length string composed of function and terminal symbols. The function symbols are represented by some operation symbols, such as $\{+, /, \sin\}$. The terminal symbols are some problem-specific attributes, such as the processing time δ , the number of remaining unscheduled operations ξ , and the remaining processing time ϑ . Through the transformation process described in Fig. 5, each chromosome in GEP can be converted to a specific dispatching rule.

$\{\delta, \xi, \vartheta\}$ is some common terminal symbols extensively adopted for constructing dispatching rules, such as the longest processing time (LPT) and the least total work remaining (TWK). However, these common terminal symbols ignore assembly features and LS attributes. As a result, the dispatching rule constructed by common terminal symbols may be ineffective for the studied MAJSP-LS. In this regard, two new terminal symbols are introduced. The first one is the assigned assembly station of a component v . The component assigned to the previous station is assembled first. The second one is the subplot feature η . If two adjacent sublots come from the same lot, η takes 1; otherwise, η takes 0. In sum, the adopted terminal symbols are $\{\delta, \xi, \vartheta, v, \eta\}$, and the selected function symbols are $\{+, -, *, /\}$. For each chromosome, its head is formed by function and terminal symbols, while its tail only contains terminal symbols.

By randomly selecting the function and terminal symbols, diverse chromosomes can be created. To find a well-performed chromosome, three evolution operators including the tournament selection, mutation and recombination operators [48] are adopted. Algorithm 1 shows the rule mining procedure of GEP. In algorithm 1, the fitness of each chromosome is evaluated by the following steps. (1) transform the chromosome into a rule. (2) create one solution for each MAJSP-LS calibration case via the rule-based initialization in Section 5.2. (3) calculate the fitness of each solution via the decoding method in Section 5.2. (4) take the average fitness of all cases as the final fitness of the chromosome.

Algorithm 1. The pseudocode of implementing GEP.

Input: Function and terminal symbols, MAJSP-LS calibration dataset.

Output: Sublot sequence rule.

01: Population initialization.

02: **While** the training round is not reached

03: Convert each chromosome into a rule.

04: Create one solution for each MAJSP-LS calibration case.

05: Calculate the average fitness of each chromosome.

06: Update the population by the selection, mutation and recombination operators.

07: **End while**

08: Find the best chromosome, and convert it into a rule.

Through Algorithm 1, a subplot sequence rule, $\min(v - \eta - \xi \bullet \xi)$ is mined. For the studied problem, $\xi \bullet \xi > v > \eta$ is generally satisfied. Thus, the rule means to first select the subplot with the most unscheduled operations, then select the subplot which needs to be first assembled, and then select the subplot which belongs to the same lot as the previous subplot. Note that, if multiple sublots have identical $v - \eta - \xi \bullet \xi$, they are selected according to the assembly sequence, and the subplot with a smaller index is selected first.

5.4. Neighborhood operators

For the studied problem, three types of neighborhood operators are designed.

Type one is subplot split mutation. It randomly selects one gene from the subplot split segment, and then increases or reduces the subplot quantity of the gene by one.

Type two is actual subplot oriented swap and insertion, and multi-swap and multi-insertion. The subplot sequence segment consists of actual and invalid sublots. Changing the position of invalid sublots cannot influence fitness. Therefore, the actual subplot oriented swap operator randomly selects two actual sublots and exchanges their positions. The actual subplot oriented insertion randomly selects one actual subplot and an insertion position, and then inserts the subplot into the selected position. The multi-swap and multi-insertion execute the swap and insertion operators I times, and at least one of the selected sublots or insertion positions is from the actual subplot fragment. Here, I denotes the number of product types.

Type three includes product swap, product insertion and product flip. The swap and interion operators work similarly to those in type two. The product flip randomly selects one fragment from the assembly sequence segment and inverts the order of the products within the fragment.

5.5. Adaptive selection of neighborhood operators

Section 5.4 defines eight neighborhood operators. These operators may have different performances on different running stages of the evolution. In most studies [44,49], the performance difference among neighborhood operators is ignored and neighborhood operators are randomly selected to generate neighbor solutions. To enhance the exploitation ability, an adaptive selection mechanism is developed to select neighborhood operators according to their historical performances. The core of the selection mechanism is to calculate the cumulative score of each neighborhood operator and then apply roulette wheel selection to select the high-scoring neighborhood operator with a higher probability.

The initial scores of all neighborhood operators are set to one. In each iteration, the fitness values of the neighbor solutions created by all the selected neighborhood operators are compared and the neighborhood operator with the best fitness is marked. If the obtained best fitness is smaller than the fitness of the incumbent solution, the score of the marked neighborhood operator increases by K but the maximum score cannot exceed $N \max$. In addition, considering that the generation of neighbor solutions consumes a certain amount of computing resources, the scores of all the current selected neighborhood operators are subtracted by a small value S , but the minimum score is no less than $N \min$. Through preliminary experiments, $N \max$, $N \min$, K and S respectively take $3 \cdot NO$, 1 , $0.5 \cdot NO$ and 0.1 , where NO denotes the number of

neighborhood operators.

In roulette wheel selection, the selection probability of each neighborhood operator is determined by the ratio of the score of each neighborhood operator to the total score. The higher the score of the neighborhood operator, the more chances of being selected.

5.6. Adaptive adjustment of neighborhood size

Neighborhood size is a key parameter that affects the performance of algorithms. A large neighborhood size may cover more landscapes and detect more promising neighbor solutions. However, more computing resources are required for the generation and evaluation of neighbor solutions. As for the small neighborhood size, its effect is exactly opposite to that of the large neighborhood size.

Instead of setting a fixed neighborhood size or only creating one neighbor as in most studies [3,50], an adaptive adjustment mechanism is designed to adjust the neighborhood size according to the running state of algorithms. Here, the running state is evaluated by (CT/ST) , the ratio of the algorithm's cumulative running time to a given stopping time. $(CT/ST) \in [0, 1]$, where (CT/ST) close to zero means that the algorithm is in the early stage of running; conversely, in the late stage. Through Eq.(37), a large neighborhood size is set in the early stage, which is beneficial to search for more potential solutions. With the increase of the cumulative running time, the neighborhood size is reduced linearly to promote convergence.

$$NS = 1 + \text{round}(BNS \bullet (1 - CT/ST)) \quad (37)$$

where CT denotes the cumulative running time; ST is the given stopping time of algorithms; BNS is a fixed basic neighborhood size.

5.7. Acceptance mechanism

In ASA, the classical Metropolis acceptance criterion [51] is applied to accept the newly created solutions. In brief, if the newly created solution performs better than the incumbent solution, the new solution is used to replace the incumbent solution; otherwise, the new solution is accepted to replace the incumbent solution with a probability P . P is calculated by Eq.(38),

$$P = \exp\left(-\frac{f_n - f_c}{T}\right) \quad (38)$$

where T is the current temperature. f_n and f_c denote the fitness of the new solution and the incumbent solution.

5.8. Adaptive control of temperature

ASA is sensitive to temperature. The lower the temperature, the smaller the probability of accepting inferior solutions. If inferior solutions are always accepted in a small probability, ASA is easy to fall into local optimal; otherwise, ASA is hard to converge. To enhance ASA, Q-learning is applied to adaptively control the temperature according to the evolution state of ASA.

Q-learning is a widely used reinforcement learning algorithm that can find the best series of actions according to the agent's state. In this work, the state is represented by the evolution state of ASA and the action is to select different temperature control factors. An agent observes the current state s_t , and moves to a next state s_{t+1} by taking an

Table 7
Statistical results of the Friedman test.

Friedman test	IABC	GA-DR	TAVNS	TS	EICA	ASA	p-value
Rank of average <i>RPD</i>	5.91	3.41	4.19	2.44	4.06	1	7.33E-26
Rank of minimum <i>RPD</i>	5.72	5	3.25	2.25	3.66	1.13	7.99E-27

action a_t based on ϵ – greedy strategy [20]. Meanwhile, a reward r is returned for the selected action, and a Q-table which stores the Q-value $Q_t(s_t, a_t)$ of all the state-action pairs is updated by Eq.(39). By iteratively improving the selection of the agent, a final Q-table is well-trained. From the Q-table, the best series of actions can be found.

$$Q_{t+1}(s_t, a_t) \leftarrow Q_t(s_t, a_t) + \lambda \cdot \left[r_t + \gamma \cdot \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right] \quad (39)$$

where λ and γ are the learning rate and discount factor.

In the following, the problem-specific state, action and reward are first designed, and then the training procedure of the Q-table is presented in Fig. 6.

State: The evolution state of ASA is represented by the cumulative running time (Cr) and the number of consecutive failing updates of the best-so-far solution (Fu). To speed up the convergence of Q-learning, Cr and Fu are discretized into several subintervals. Cr consists of three subintervals: $\{[0, ST/3], [ST/3, ST/1.5], [ST/1.5, ST]\}$. Here, ST denotes the given stopping time of ASA. Fu consists of five subintervals: $\{[0, 5], [5, 20], [20, 40], [40, 60], [60, \infty]\}$. The combination of Cr and Fu has 15 groups, and each group represents a state.

Action: In each iteration, one of the five temperature control factors is selected to adjust the temperature. They are $\{0.95, 0.97, 0.99, 1, 1.02\}$. The temperature at the next iteration is equal to the current temperature multiplied by the selected temperature control factor.

Reward: To encourage a good action and punish an unprofitable action, the reward is set as the change value of the best-so-far solution obtained by the ASA before and after temperature adjustment.

6. Experiment study

This section aims to evaluate the effectiveness of the proposed algorithm. To achieve this, multiple comparison algorithms are programmed with MATLAB language, and then numerous comparison experiments are designed and implemented on a processor with Intel (R) Core (TM) i7–10700 CPU. For each run, the stopping time of all comparison algorithms is set as $I \cdot N \cdot 10$ s. Here, I represents the number of product types, and N denotes the number of components of a product. The relative percentage deviation (RPD) is employed as the performance metric. RPD is calculated by Eq.(40). The algorithm with a smaller RPD performs better.

$$RPD = (f_{ci} - f_{cb}) / f_{cb} \quad (40)$$

where f_{ci} and f_{cb} denote the fitness obtained by algorithm i and the best fitness by all algorithms.

In the work by [3], a MAJSP dataset including six instances A1–A6 for parameter calibration and thirty-two instances B1–B32 for comparative experiments are created. To create testing instances for the studied MAJSP-LS, some lot related parameters are created and appended to the existing MAJSP instances. In detail, the lot size of each product type is created via the uniform distribution U [4,8], and the setup time between sublots obeys U [10,30]. The manufacturing cost and setup cost coefficients are subject to U [10,20] and U [30,50], respectively.

6.1. Parameter calibration and Q-table training

In this section, key parameters of all comparison algorithms are calibrated through the Taguchi method. For the proposed ASA, it needs to further implement Q-learning on the SA-OS (the ASA without the Q-learning-based temperature adaptive control mechanism) to adaptively control the temperature. Thus, the parameters of the SA-OS are calibrated first, and then the best parameter combination is applied to the ASA to train Q-learning.

Regarding the SA-OS, four parameters including the initial temperature (IT), cooling rate (CR), iteration number under each temperature (IC), and basic neighborhood size (BNS) are calibrated. Each parameter

is tested at four levels, and a Taguchi orthogonal $L_{16}(4^4)$ is considered. For each parameter combination, the SA-OS is tested on six calibration cases, and the average RPD of these cases is calculated and utilized as the response value. Based on the obtained response values, the main effect plot for signal-to-noise ratio (SNR) is drawn as shown in Fig. 7 to explore the trends of parameter levels. Here, SNR is calculated by Eq.(41) and used to identify the parameter combination that minimizes the effect of noise on the response [52]. The bigger the SNR is, the better the robustness is. Thus, the recommended parameter combination of the SA-OS is $\{IT = 1000; CR = 0.97; IC = 40; BNS = 10\}$.

$$SNR = -10 \cdot \log_{10} \left(\frac{\sum_{i=1}^n (y_i)^2}{n} \right) \quad (41)$$

where y_i denotes the response value of the i -th experiment for each parameter at the same level, n is the number of experiments.

Following the above procedure, the parameters of all comparison algorithms are calibrated and the final selected parameter combinations are listed in Tables 1–2.

After the ideal parameters of the ASA are determined, a Q-table is trained according to the training procedure in Fig. 6. The selected training case is the largest calibration case A6 and the stopping time of each training round is $I \cdot N \cdot 10$ s. As for the three basic parameters of Q-learning: learning rate λ , greedy factor ϵ and discount factor γ , they take the same value as those in the literature [20]. Finally, through forty rounds, a well-performed Q-table is obtained and given in Table 3. For a given evolution state of the ASA, the best temperature control action is the one corresponding to the highest Q-value. Considering that the Q-learning model may be overfitted, the best action is selected with a probability of 0.9, and the other actions are randomly selected with a probability of 0.1 in the following testing experiment.

6.2. Performance evaluation of the extracted subplot sequence rule

To evaluate the effectiveness of the subplot sequence rule mined by GEP, a rule comparison experiment is designed by comparing the discovered rule with the random rule and several widely used dispatching rules. The involved dispatching rules include SPT (the operation with the shortest processing time has a higher priority), LPT (the operation with the longest processing time has a higher priority), TWK (first select the operation belonging to the subplot with the least total work remaining) and MTWK (first select the operation belonging to the subplot with the most total work remaining).

The above six rules are tested on 32 testing cases (B1–B32). For each case, initial solutions are created by adopting one of the above six rules to create the subplot sequence and applying the initialization method in Section 5.2 to create the subplot split and assembly sequence. For a fair comparison, each rule runs 30 times on each case to create 30 initial solutions. Then, the decoding method in Section 5.2 is applied to obtain the average and minimum fitness of 30 initial solutions. Table 4 lists the testing results.

From Table 4, it can be observed that in almost all cases, the fitness values obtained by the proposed rule are much smaller than those by other rules. This indicates that the proposed rule is conducive to finding high-quality initial solutions. In addition, except for the MTWK rule, the performance of other widely used dispatching rules is far from satisfactory and even worse than that of the random rule. This means that the dispatching rules effective for other scheduling problems are not necessarily applicable to the studied MAJSP-LS. As for the MTWK rule, its overall performance is only inferior to that of the proposed rule. The reason is that both the MTWK rule and the proposed rule demand that the component with the most remaining operations is scheduled first. Compared with the MTWK rule, the proposed rule further considers the assembly and lot features of components, which makes the proposed rule more adaptive to the studied MAJSP-LS.

6.3. Effectiveness of the adaptive neighborhood search

The adaptive neighborhood search contains an adaptive selection mechanism of neighborhood operators and an adaptive adjustment mechanism of neighborhood size. To verify the effectiveness of the two adaptive mechanisms, four variants of ASA are designed: SA, SA-O, SA-S and SA-OS. Among them, SA selects neighborhood operators randomly, and the neighborhood size is fixed during the evolution. Different from SA, SA-O adaptively selects neighborhood operators, while SA-S adaptively adjusts the neighborhood size during the evolution. SA-OS simultaneously adopts the above two adaptive mechanisms. For a fair comparison, all the variants adopt the same encoding, decoding, rule-based initialization, neighborhood operators and acceptance mechanism.

The four variants are tested on 32 cases with each case tested 30 independent runs. During each run, once the cumulative running time reaches the given stopping time $I \bullet N \bullet 10$ s, the current best fitness is printed. Then, the average and minimum fitness on 30 runs of each algorithm in each case are calculated and recorded in Table 5.

In Table 5, the best results are shown in bold. It can be seen that most of the best results are located in the variant algorithm with two adaptive mechanisms, while only two best results are located in the variant algorithm without the adaptive mechanism. These indicate that introducing the proposed adaptive mechanism can improve the performance of algorithms. To analyze the performance deviation between the solutions obtained by each algorithm and the best solutions obtained by all algorithms, the corresponding average and minimum *RPD* of each algorithm on each case are calculated based on the data in Table 5. To display the *RPD* results more intuitively, two interval plots are presented as shown in Fig. 8. For the *RPD* metric, the smaller the better. Thus, the SA-OS performs the best, followed by the SA-S and SA-O, and the SA is the worst.

The above results are due to two factors. (1) The adaptive selection mechanism enables the neighborhood operator with good performance to be preferentially selected to generate neighbor solutions, which can increase the chance of finding potential solutions. (2) The adaptive adjustment mechanism of neighborhood size can make a good compromise between computing resources and the coverage of neighborhood space to maximize the utilization rate of computing resources.

6.4. Effectiveness of the adaptive control of temperature

To evaluate the effectiveness of the adaptive control of temperature, the SA-OS is compared with ASA (the variant of SA-OS with a Q-learning-based temperature control mechanism). Similar to Section 6.3, the two comparison algorithms are tested on 32 cases, and the *RPD* metric is calculated to evaluate their performance. Fig. 9 and Fig. 10 show the comparison results.

As observed in Fig. 9 and Fig. 10, the *RPD* values obtained by the ASA are smaller than those by the SA-OS in most cases. As for the remaining cases, the results obtained by the ASA are close to the best ones. All these suggest that the Q-learning-based temperature control mechanism is helpful for improving the performance of the ASA. The most likely reason is that by controlling the temperature, the acceptance probability of inferior solutions can be effectively adjusted according to the evolution state, so as to help the algorithm timely jump out of the local optimal and move towards the global optimal.

6.5. Comparison with other algorithms

To further verify the effectiveness of the proposed ASA in solving the studied MAJSP-LS, several representative algorithms which are originally designed for those problems similar to the MAJSP-LS are reproduced and used for comparison. These algorithms are classified into two types. Type one includes tabu search (TS) [28] and enhanced imperialist competitive algorithm (EICA) [42] for the JSP with LS. Type two

includes a two-level self-adaptive variable neighborhood search (TAVNS) [15], improved artificial bee colony (IABC) [4] and genetic algorithm-dispatching rule (GA-DR) method [32] for the assembly job-shop scheduling with LS. Especially, the GA-DR method is selected since it was first developed to handle the assembly job-shop scheduling with LS. For the GA-DR, GA is employed to optimize the assembly sequence and subplot split, and the extracted subplot sequence rule is adopted to determine the subplot sequence. As for the TS and TAVNS, they are the two most commonly used single-solution-based algorithms besides the SA, and hence can be employed to measure the local search ability of the ASA.

Similar to Section 6.3, under the same stopping time, all algorithms are tested for 30 runs on each case to obtain the average and minimum fitness. Table 6 lists the comparison results. It can be seen that in terms of average fitness, the proposed ASA obtains the smallest values in all cases. Regarding the minimum fitness, except for three cases, the results of the ASA are slightly worse than those of TAVNS/TS, and in the other twenty-nine cases, the best values are located in the ASA. These results indicate that compared with other algorithms, the proposed algorithm can find more satisfactory scheduling plans in which the completion time is shorter and the setup times are lower.

To statistically analyze the differences between the comparison algorithms, a non-parametric Friedman test [53] is designed. The six comparison algorithms are viewed as the test variables. Based on the data in Table 6, the average and minimum *RPD* metrics are calculated and used as the response values. Table 7 shows the test results. It can be seen that under the *p*-value less than 0.001, the proposed ASA gets the minimum rank in *RPD*, followed by the TS, EICA, TAVNS, GA-DR and IABC. These indicate that the proposed algorithm outperforms other algorithms with statistical significance. The above results are attributed to the high-quality initialization provided by the extracted rule, and multiple adaptive strategies extracted from a series of state information.

7. Conclusion and future work

This paper addresses the mixed-model assembly job-shop scheduling problem with lot streaming. To achieve the minimization of manufacturing and setup costs, a MILP model and an adaptive simulated annealing algorithm are developed. In the developed MILP model, five types of constraints including sequence-dependent setup constraints, sequence constraints of components within a subplot and mixed-model assembly constraints are formulated. Two types of linearization techniques are designed to eliminate the non-linearity caused by lot streaming and sequence-dependent setup. Regarding the proposed algorithm, one subplot sequence rule and three adaptive mechanisms are designed. In detail, subplot attributes and assembly features are extracted and imported into GEP to construct subplot sequence rule; the evolution state of the algorithm is learned by Q-learning to adaptively control the temperature; the historical performance of neighborhood operators is evaluated and used to adaptively select neighborhood operators; the running state of the algorithm is employed to adjust the neighborhood size adaptively.

Finally, four groups of experiments are designed to evaluate the discovered rule, the designed adaptive mechanisms and the enhanced algorithm. The experiment results indicate that both the discovered rule and the adaptive mechanisms are effective, and the enhanced algorithm significantly outperforms other comparison algorithms. The reason for the above promising results is that the discovered rule takes full account of and the designed adaptive mechanisms make full use of three types of state information to guide the evolution.

In the future, other features such as transportation time, part sharing and energy consumption can be considered to improve the application of the proposed model. In addition, other performance criteria including the work-in-process inventory and due date can be considered.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (No. 52275504).

References

- [1] Framinan JM, Perez-Gonzalez P, Fernandez-Viagas V. Deterministic assembly scheduling problems: a review and classification of concurrent-type scheduling models and solution procedures. *Eur J Oper Res* 2018;273:401–17. <https://doi.org/10.1016/j.ejor.2018.04.033>.
- [2] Zhang BK, Xu LY, Zhang J. A multi-objective cellular genetic algorithm for energy-oriented balancing and sequencing problem of mixed-model assembly line. *J Clean Prod* 2020;244:118845. <https://doi.org/10.1016/j.jclepro.2019.118845>.
- [3] Cheng LX, Tang QH, Zhang LP, et al. Inventory and total completion time minimization for assembly job-shop scheduling considering material integrity and assembly sequential constraint. *J Manuf Syst* 2022;65:660–72. <https://doi.org/10.1016/j.jmsy.2022.10.013>.
- [4] Li X, Lu J, Yang C, et al. Research of flexible assembly job-shop batch-scheduling problem based on improved artificial bee colony. *Front Bioeng Biotech* 2022;10:909548. <https://doi.org/10.3389/fbioe.2022.909548>.
- [5] Zheng P, Zhang P, Wang JL, et al. A data-driven robust optimization method for the assembly job-shop scheduling problem under uncertainty. *Int J Comput Integ M* 2020;35:1–16. <https://doi.org/10.1080/0951192x.2020.1803506>.
- [6] Liao CJ, Lee CH, Lee HC. An efficient heuristic for a two-stage assembly scheduling problem with batch setup times to minimize makespan. *Comput Ind Eng* 2015;88:317–25. <https://doi.org/10.1016/j.cie.2015.07.018>.
- [7] Victor FV, Carla T, Jose MF. Assembly flowshop scheduling problem: Speed-up procedure and computational evaluation. *Eur J Oper Res* 2022;299:869–82. <https://doi.org/10.1016/j.ejor.2021.10.001>.
- [8] Sara H, Rubén R, Carlos AR. Heuristics and metaheuristics for the distributed assembly permutation flowshop scheduling problem with sequence dependent setup times. *Int J Prod Econ* 2015;169:76–88. <https://doi.org/10.1016/j.ijpe.2015.07.027>.
- [9] Zhang SC, Wang SY. Flexible assembly job-shop scheduling with sequence-dependent setup times and part sharing in a dynamic environment: constraint programming model, mixed-integer programming model, and dispatching rules. *IEEE T Eng Manage* 2018;65:487–504. <https://doi.org/10.1109/TEM.2017.2785774>.
- [10] Wong TC, Ngan SC. A comparison of hybrid genetic algorithm and hybrid particle swarm optimization to minimize makespan for assembly job shop. *Appl Soft Comput* 2013;13:1391–9. <https://doi.org/10.1016/j.asoc.2012.04.007>.
- [11] Fattahi P, Bagheri Rad N, Daneshamooz F, et al. A new hybrid particle swarm optimization and parallel variable neighborhood search algorithm for flexible job shop scheduling with assembly process. *Assembly Autom* 2020;40:419–32. <https://doi.org/10.1108/AA-11-2018-0178>.
- [12] Wu X, Liu X, Zhao N. An improved differential evolution algorithm for solving a distributed assembly flexible job shop scheduling problem. *Memet Comput* 2019;11:335–55. <https://doi.org/10.1007/s12293-018-00278-7>.
- [13] Wang H, Sarker BR, Li J, et al. Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning. *Int J Prod Res* 2021;59:5867–83. <https://doi.org/10.1080/00207543.2020.1794075>.
- [14] Ren WB, Wen JQ, Yan Y, et al. Multi-objective optimisation for energy-aware flexible job-shop scheduling problem with assembly operations. *Int J Prod Res* 2021;59:7216–31. <https://doi.org/10.1080/00207543.2020.1836421>.
- [15] Fatemeh D, Parviz F, Seyed MHH. Scheduling in a flexible job shop followed by some parallel assembly stations considering lot streaming. *ENG Optim* 2022;54:614–33. <https://doi.org/10.1080/0305215X.2021.1887168>.
- [16] Zhang SC, Li X, Zhang BW, et al. Multi-objective optimisation in flexible assembly job shop scheduling using a distributed ant colony system. *Eur J Oper Res* 2020;283:441–60. <https://doi.org/10.1016/j.ejor.2019.11.016>.
- [17] Sheikh S, Komaki GM, Kayvanfar V, et al. Multi-Stage assembly flow shop with setup time and release time. *Oper Res Perspect* 2019;6:100111. <https://doi.org/10.1016/j.orp.2019.100111>.
- [18] Komaki GM, Teymourian E, Kayvanfar V, et al. Improved discrete cuckoo optimization algorithm for the three-stage assembly flowshop scheduling problem. *Comput Ind Eng* 2017;105:158–73. <https://doi.org/10.1016/j.cie.2017.01.006>.
- [19] Zhang ZK, Tang QH, Chica M. Maintenance costs and makespan minimization for assembly permutation flow shop scheduling by considering preventive and corrective maintenance. *J Manuf Syst* 2021;59:549–64. <https://doi.org/10.1016/j.jmsy.2021.03.020>.
- [20] Cheng LX, Tang QH, Zhang LP, et al. Scheduling flexible manufacturing cell with no-idle flow-lines and job-shop via Q-learning-based genetic algorithm. *Comput Ind Eng* 2022;169:108293. <https://doi.org/10.1016/j.cie.2022.108293>.
- [21] Cheng M, Mukherjee NJ, Sarin SC. A review of lot streaming. *NT J Prod Res* 2013;51:7023–46. <https://doi.org/10.1080/00207543.2013.774506>.
- [22] Lalitha JL, Mohan N, Pillai VM. Lot streaming in $[N-1](1)+N(m)$ hybrid flow shop. *J Manuf Syst* 2017;44:12–21. <https://doi.org/10.1016/j.jmsy.2017.04.018>.
- [23] Gürsoy Yılmaz B, Faruk Yılmaz Ö. Lot streaming in hybrid flowshop scheduling problem by considering equal and consistent sublots under machine capability and limited waiting time constraint. *Comput Ind Eng* 2022;173:108745. <https://doi.org/10.1016/j.cie.2022.108745>.
- [24] Novas JM. Production scheduling and lot streaming at flexible job-shops environments using constraint programming. *Comput Ind Eng* 2019;136:252–64. <https://doi.org/10.1016/j.cie.2019.07.011>.
- [25] Chan FTS, Wong TC, Chan LY. The application of genetic algorithms to lot streaming in a job-shop scheduling problem. *Int J Prod Res* 2009;47:3387–412. <https://doi.org/10.1080/00207540701577369>.
- [26] Buscher U, Shen L. An integrated tabu search algorithm for the lot streaming problem in job shops. *Eur J Oper Res* 2009;2:385–99. <https://doi.org/10.1016/j.ejor.2008.11.046>.
- [27] Lei DM, Guo XP. Scheduling job shop with lot streaming and transportation through a modified artificial bee colony. *Int J Prod Res* 2013;51:4930–41. <https://doi.org/10.1080/00207543.2013.784404>.
- [28] Bozek A, Werner F. Flexible job shop scheduling with lot streaming and sublot size optimisation. *Int J Prod Res* 2018;56:6391–411. <https://doi.org/10.1080/00207543.2017.1346322>.
- [29] Xu XL, Li L, Fan LX, et al. Hybrid discrete differential evolution algorithm for lot splitting with capacity constraints in flexible job scheduling. *Math Probl Eng* 2013;2013:1–10. <https://doi.org/10.1155/2013/986218>.
- [30] Defersha FM, Chen MY. Jobshop lot streaming with routing flexibility, sequence-dependent setups, machine release dates and lag time. *Int J Prod Res* 2012;50:2331–52. <https://doi.org/10.1080/00207543.2011.574952>.
- [31] Vuong VH, Do TKA, Hoang NN. Linear programming assisted (not embedded) genetic algorithm for flexible jobshop scheduling with lot streaming. *Comput Ind Eng* 2018;117:319–35. <https://doi.org/10.1016/j.cie.2018.02.010>.
- [32] Chan FTS, Wong TC, Chan LY. Lot streaming for product assembly in job shop environment. *Robot CIM-Int Manuf* 2008;24:321–31. <https://doi.org/10.1016/j.rcim.2007.01.001>.
- [33] Chan FTS, Wong TC, Chan LY. An evolutionary algorithm for assembly job shop with part sharing. *Comput Ind Eng* 2009;57:641–51. <https://doi.org/10.1016/j.cie.2008.11.017>.
- [34] Panwalkar SS, Iskander W. A survey of scheduling rules. *Oper Res* 1977;25:45–61. <https://doi.org/10.1287/opre.25.1.45>.
- [35] Zhang L, Tang Q, Wu Z, et al. Mathematical modeling and evolutionary generation of rule sets for energy-efficient flexible job shops. *Energy* 2017;138:210–27. <https://doi.org/10.1016/j.energy.2017.07.005>.
- [36] Zhang F, Su N, Yi M, et al. In: Kay Chen Tan, Dacheng Tao, editors. *Genetic Programming for Production Scheduling*. Singapore: Springer; 2021. p. 1–357.
- [37] Gil-Gala FJ, Đurašević M, Varela R, et al. Ensembles of priority rules to solve one machine scheduling problem in real-time. *Inf Sci* 2023;634:340–58. <https://doi.org/10.1016/j.ins.2023.03.114>.
- [38] Zhang F, Mei Y, Nguyen S, et al. Survey on genetic programming and machine learning techniques for heuristic design in job shop scheduling. *IEEE T Evol Comput* 2023;1–21. <https://doi.org/10.1109/tevc.2023.3255246>.
- [39] Guo W, Lei Q, Song Y, et al. A learning interactive genetic algorithm based on edge selection encoding for assembly job shop scheduling problem. *Comput Ind Eng* 2021;159. <https://doi.org/10.1016/j.cie.2021.107455>.
- [40] Sukkerd W, Wuttipornpun T. Hybrid genetic algorithm and tabu search for finite capacity material requirement planning system in flexible flow shop with assembly operations. *COMPUT IND ENG* 2016;97:157–69. <https://doi.org/10.1016/j.cie.2016.05.006>.
- [41] Güçdemir H, Selim H. Customer centric production planning and control in job shops: a simulation optimization approach. *J Manuf Syst* 2017;43:100–16. <https://doi.org/10.1016/j.jmsy.2017.02.004>.
- [42] Li Y, Yang Z, Wang L, et al. A hybrid imperialist competitive algorithm for energy-efficient flexible job shop scheduling problem with variable-size sublots. *Comput Ind Eng* 2022;172. <https://doi.org/10.1016/j.cie.2022.108641>.
- [43] Sun M, Cai Z, Zhang H. A teaching-learning-based optimization with feedback for L-R fuzzy flexible assembly job shop scheduling problem with batch splitting. *Expert Syst Appl* 2023;224:120043. <https://doi.org/10.1016/j.eswa.2023.120043>.
- [44] Akram K, Kamal K, Zeb A. Fast simulated annealing hybridized with quenching for solving job shop scheduling problem. *Appl Soft Comput* 2016;49:510–23. <https://doi.org/10.1016/j.asoc.2016.08.037>.
- [45] Wu CC, Azzouz A, Chung IH, et al. A two-stage three-machine assembly scheduling problem with deterioration effect. *Int J Prod Res* 2019;57:6634–47. <https://doi.org/10.1080/00207543.2019.1570378>.
- [46] Nejati M, Mahdavi I, Hassanzadeh R, et al. Lot streaming in a two-stage assembly hybrid job shop scheduling problem with a work shift constraint. *J Ind Prod Eng* 2016;33:459–71. <https://doi.org/10.1080/21681015.2015.1126653>.
- [47] Defersha FM, Obimuyiwa D, Yimer AD. Mathematical model and simulated annealing algorithm for setup operator constrained flexible job shop scheduling problem. *Comput Ind Eng* 2022;171. <https://doi.org/10.1016/j.cie.2022.108487>.

- [48] Zhang LP, Li ZX, Krolczyk G, et al. Mathematical modeling and multi-attribute rule mining for energy efficient job-shop scheduling. *J Clean Prod* 2019;241:1–14. <https://doi.org/10.1016/j.jclepro.2019.118289>.
- [49] Yağmur E, Kesen SH. Multi-trip heterogeneous vehicle routing problem coordinated with production scheduling: memetic algorithm and simulated annealing approaches. *Comput Ind Eng* 2021;161:107649. <https://doi.org/10.1016/j.cie.2021.107649>.
- [50] Ceschia S, Di Gaspero L, Schaerf A. Solving discrete lot-sizing and scheduling by simulated annealing and mixed integer programming. *Comput Ind Eng* 2017;114: 235–43. <https://doi.org/10.1016/j.cie.2017.10.017>.
- [51] Ying K, Lin S. Solving no-wait job-shop scheduling problems using a multi-start simulated annealing with bi-directional shift timetabling algorithm. *Comput Ind Eng* 2020;146:106615. <https://doi.org/10.1016/j.cie.2020.106615>.
- [52] Tirkolaee EB, Goli A, Faridnia A, et al. Multi-objective optimization for the reliable pollution-routing problem with cross-dock selection using Pareto-based algorithms. *J Clean Prod* 2020;276:1–23. <https://doi.org/10.1016/j.jclepro.2020.122927>.
- [53] Cheng LX, Tang QH, Zhang LP, et al. Mathematical model and enhanced cooperative co-evolutionary algorithm for scheduling energy-efficient manufacturing cell. *J Clean Prod* 2021:326. <https://doi.org/10.1016/j.jclepro.2021.129248>.